

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
Escuela Profesional de Ingeniería de Software



Lenguaje de Programación II (Orientado a Objetos)

SW403-U

Prototipo de Videoconferencia

Integrantes:

Velarde Timoteo Alejandro Alfredo

Bustamante Rojas Luis

Medrano Chero Michell

Docente:

Cisneros Napravnik Jan Eduardo

Lima, 2026

Índice

INTRODUCCIÓN	4
Contexto y Justificación	4
Objetivos del Proyecto	4
Alcance y Limitaciones Académicas	4
REQUERIMIENTOS	6
Requerimientos Funcionales:	6
Requerimientos No Funcionales:	6
ARQUITECTURA	8
Estilo Arquitectónico y Descomposición en Capas	8
Componentes del Lado del Cliente (Capa de Presentación y Aplicación)	9
Componentes del Lado del Servidor (Capa de Control y Enrutamiento)	10
Capa de Persistencia y Almacenamiento Estático (Capa de Datos)	11
Modelado de Interacción y Flujo de Datos Distribuidos	12
DISEÑO DE BASE DE DATOS Y PERSISTENCIA	14
Motor Relacional y Enfoque Físico	14
Modelo Entidad-Relación y Diccionario de Datos	14
Restricciones e Integridad Referencial	16
Script de Definición de Datos (DDL)	16
Inserción de Datos Base (Seed Data)	18
PROTOCOLO DE COMUNICACIÓN POR SOCKETS (FRAMING)	19
Justificación del Modelo de Enmarcado (Framing)	19
Estructura de la Trama de Red	19
Tipos de Mensajes y Serialización JSON	20
IMPLEMENTACIÓN DE MÓDULOS	22
Módulo de Login e Identidad	22
Módulo de Salas y Sala de Espera (Control de Acceso)	23
Módulo de Chat de Texto	24
Módulo de Transferencia de Documentos (I/O en Bloques)	25
Módulo de Captura y Transmisión de Cámara (Procesamiento de Imágenes)	26
PRUEBAS Y CASOS DE USO	27
Pruebas de Autenticación y Control de Sesión	27

Pruebas de Salas y Gestión de Concurrencia	29
Pruebas de Interacción Asíncrona (Chat y Multimedia)	31
Pruebas de Control Administrativo (Opcionales / Mejoras)	34
CONCLUSIONES Y MEJORAS FUTURAS	37
Conclusiones	37
Dificultades Encontradas y Limitaciones del Prototipo	38
Mejoras Futuras Recomendadas.....	38
ANEXOS	40

INTRODUCCIÓN

Contexto y Justificación

El presente informe documenta el análisis, diseño y construcción de un sistema de videoconferencia prototípico basado en una arquitectura cliente-servidor. El problema central que este proyecto resuelve es la implementación de un entorno de comunicación concurrente bidireccional en tiempo real, prescindiendo de librerías de alto nivel para el transporte multimedia y construyendo, en su lugar, una solución fundamentada en sockets TCP puros. Esta aproximación obliga a gestionar manualmente el flujo de bytes, la concurrencia de usuarios y la sincronización de estados distribuidos.

Objetivos del Proyecto

El objetivo principal es demostrar empíricamente los fundamentos de las redes de computadoras y la arquitectura de software. De manera específica, el sistema persigue:

- Implementar un canal de comunicación confiable para el intercambio de metadatos (JSON) y carga binaria (archivos y frames de video) mediante un protocolo de enmarcado (framing) propio.
- Desarrollar un sistema de control de acceso y concurrencia (salas de espera, roles de anfitrión e invitado).
- Garantizar la persistencia de información (usuarios, chats y metadatos de archivos) a través de una base de datos relacional integrada.

Alcance y Limitaciones Académicas

A diferencia de plataformas comerciales que emplean WebRTC o protocolos de datagramas (UDP) para la transmisión fluida de video y audio, el alcance de este proyecto es estrictamente académico. El diseño prioriza la fiabilidad de la entrega y el ordenamiento de los paquetes mediante TCP. En consecuencia, la transmisión de la cámara web se ha modelado de forma simplificada, capturando imágenes fijas a baja frecuencia, comprimiéndolas en formato JPG de menor resolución (320x240) y distribuyéndolas como ráfagas binarias. Esta decisión de diseño garantiza la estabilidad del socket y evita el colapso del ancho de banda local, cumpliendo así con los requerimientos pedagógicos establecidos.

REQUERIMIENTOS

Requerimientos Funcionales:

- Registro y autenticación de usuarios mediante correo y contraseña.
- Creación de salas únicas y solicitud de ingreso a las mismas.
- Sala de espera con capacidad de admisión o rechazo por parte del anfitrión (host).
- Chat de texto en tiempo real para usuarios admitidos en una sala.
- Envío y descarga de documentos compartidos entre los participantes de la sala.
- Captura, compresión y transmisión de frames de cámara a todos los integrantes activos.

Requerimientos No Funcionales:

- Comunicación cliente-servidor estricta utilizando sockets TCP.
- Persistencia de datos relacional mediante SQLite.
- Seguridad básica implementando hashing de contraseñas con el algoritmo PBKDF2.
- Manejo de hilos (threading) y colas (queues) asíncronas para evitar bloqueos en la interfaz gráfica.

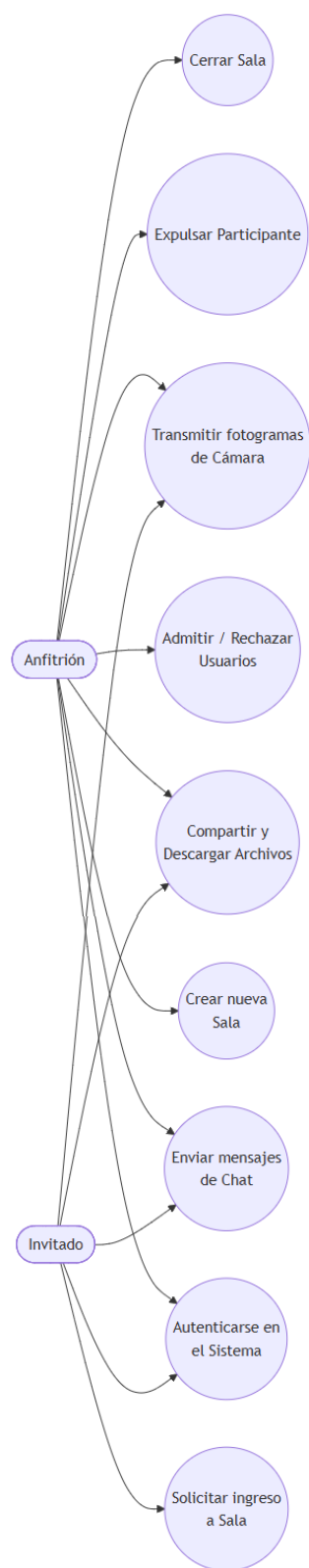


Figura 1: Diagrama de Casos de Usos

ARQUITECTURA

Estilo Arquitectónico y Descomposición en Capas

El sistema se ha diseñado bajo el estilo arquitectónico Cliente-Servidor Distribuido en Tres Capas (Multi-tier Architecture). Esta jerarquía garantiza el principio de Separación de Responsabilidades (Separation of Concerns – SoC) y el Acoplamiento Débil (Loose Coupling) entre la interfaz de usuario, la lógica de control distribuido y la capa de persistencia.

Una restricción crítica de seguridad y diseño impuesta en este sistema es el aislamiento total de la base de datos respecto al cliente. El nodo cliente carece de privilegios, credenciales o librerías para conectarse de manera directa al motor relacional; cualquier operación analítica, transaccional o de consulta debe ser obligatoriamente solicitada al servidor de sockets mediante primitivas de red bien definidas. Esto descentraliza el procesamiento crítico y protege la integridad de los datos.

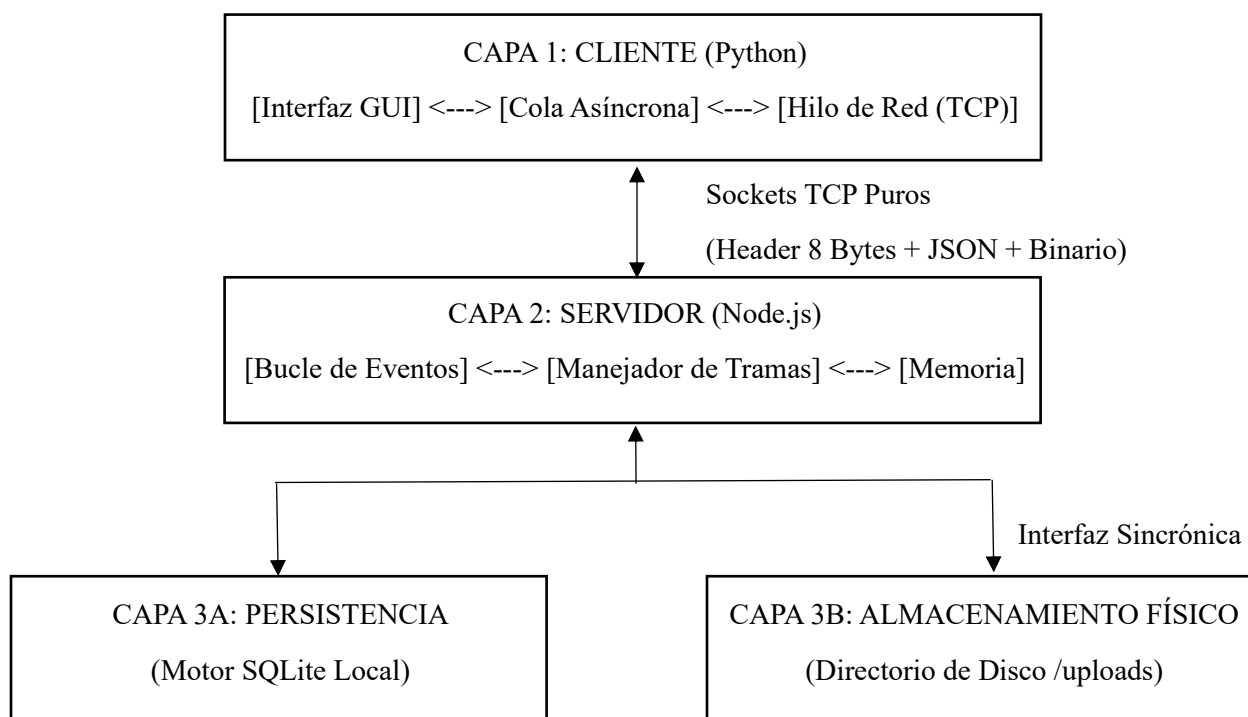


Figura 2: Diagrama de Arquitectura Lógica del Sistema

Patrones de Diseño Implementarios

Para garantizar la mantenibilidad y escalabilidad del sistema, se ha aplicado un catálogo de patrones de diseño de software:

- **Arquitecturales (MVC):** Desacoplamiento entre la capa de presentación (/UI) y la lógica (/Logic).
- **Comportamiento (Mediator y Observer):** main.py actúa como mediador para la navegación, mientras que el módulo de red utiliza callbacks (Observer) para notificar eventos asíncronos a la GUI sin bloqueos.
- **Estructurales (Facade y Adapter):** Se utiliza database.js como una Facade que encapsula el motor SQLite, y un adaptador de enmarcado (Framing) en server.js para transformar el flujo de bytes crudos de TCP en mensajes lógicos.
- **Creacionales (Singleton):** El sistema de módulos de Node.js garantiza la instanciación única de la base de datos a través de database.js.

Componentes del Lado del Cliente (Capa de Presentación y Aplicación)

El cliente (Python) ha sido refactorizado implementando una estricta separación de responsabilidades (SRP), dividiendo el sistema en dos subcapas principales para erradicar el acoplamiento:

1. **Capa Visual (Directorio /UI):** Implementa el patrón Vista. Está compuesta por módulos independientes (ui_login.py, ui_lobby.py y ui_meeting.py). Estos ficheros se encargan exclusivamente de renderizar los componentes gráficos (CustomTkinter) y capturar eventos del usuario, delegando cualquier procesamiento pesado.

2. **Capa Lógica (Directorio /Logic):** Centraliza la lógica de negocio y de transporte, actuando como Controladores:

- **network_client.py:** Encapsula la conexión TCP pura. Maneja el hilo secundario receptor (`_receive_loop`) y el mecanismo de desacoplamiento seguro (`queue.Queue`) para enviar actualizaciones a la interfaz sin congelarla.
- **video_controller.py:** Subsistema multimedia dedicado. Aísla las funciones de OpenCV para capturar fotogramas locales, redimensionar y aplicar compresión JPEG al vuelo mediante un subproceso independiente.
- **file_manager.py:** Administra las operaciones de Entrada/Salida (I/O) en disco, controlando el fraccionamiento de documentos en trozos (chunks) de 4KB.

Componentes del Lado del Servidor (Capa de Control y Enrutamiento)

El servidor (Node.js) implementa el patrón Arquitectónico de Enrutador y Controladores, resolviendo el antipatrón de lógica centralizada ("God Object") para maximizar su mantenibilidad:

1. **Núcleo de Red y Enrutador (server.js):** Actúa como el punto de entrada principal (Entry Point). Su única responsabilidad es gestionar el ciclo de vida de los sockets TCP y aplicar la lectura del protocolo de enmarcado de 8 bytes. Una vez decodificada la trama JSON, delega la carga útil al controlador correspondiente.
2. **Subcapa de Controladores (Directorio /Controllers):**
 - **authController.js:** Gestiona el ciclo de vida de las sesiones, ejecutando la validación criptográfica asíncrona de las contraseñas.

- **roomController.js:** Administra las estructuras en memoria de concurrencia, validando permisos para la creación de salas, ingresos y flujo de la sala de espera.
 - **mediaController.js:** Módulo especializado en la retransmisión masiva (Broadcast) de alto rendimiento para distribuir los frames de la cámara.
 - **fileController.js:** Controla los flujos de escritura y lectura (Streams) en el sistema de archivos del servidor para almacenar los documentos compartidos.
3. **Capa de Abstracción de Datos (database.js):** Implementa el patrón *Data Access Object* (DAO). Centraliza todas las consultas, inserciones y el hashing hacia el motor SQLite. Esto aísla a los controladores de la sintaxis SQL directa, permitiendo que la lógica de red y la base de datos evolucionen de forma independiente.

Capa de Persistencia y Almacenamiento Estático (Capa de Datos)

La base del almacenamiento está bifurcada en dos entidades controladas directamente por la capa de abstracción del servidor (database.js):

1. **Motor de Base de Datos Relacional (SQLite - node:sqlite):** Se ejecuta en modo embebido y sincrónico, lo cual elimina la latencia de red en las consultas de persistencia core. Su rol es dar estructura y permanencia histórica a las transacciones del sistema (tablas de usuarios, contraseñas hasheadas con PBKDF2, histórico cronológico de mensajes y tablas puente de control de solicitudes).
2. **Repositorio Estático de Archivos Binarios (FileSystem Local):** El disco duro del servidor aloja la carpeta /uploads. Cuando un archivo es transferido en bloques por un cliente, la base de datos únicamente almacena el identificador secuencial y el nombre físico aleatorizado con marcas de tiempo (Date.now()_NombreArchivo) para prevenir

colisiones de nombres. El archivo real es escrito directamente en disco en formato binario raw a través de flujos de escritura controlados (`fs.createWriteStream`).

Modelado de Interacción y Flujo de Datos Distribuidos

El intercambio de datos a través de la arquitectura sigue dos flujos operacionales diferenciados bajo el mismo canal de comunicación TCP, orquestados ahora por las capas lógicas de cada nodo:

- Flujo de Control Sincrónico (Petición-Respuesta):** Utilizado para transacciones críticas que requieren validación de estado (por ejemplo: `LOGIN_REQUEST` o `CREATE_ROOM`). El módulo `network_client.py` emite la petición y notifica a las vistas (/UI) para que bloqueen lógicamente sus controles interactivos. En el backend, el núcleo enruta la trama hacia el controlador correspondiente (ej. `authController.js`), el cual procesa la lógica, interactúa con la base de datos y devuelve una trama de respuesta inequívoca que libera la interfaz del cliente.
- Flujo Multimedia y de Mensajería Asíncrona (Asynchronous Pipe / Broadcast):** Utilizado para el chat, los documentos compartidos y la ráfaga de video. En este escenario, los módulos locales (`video_controller.py` y `file_manager.py`) despachan datos de manera continua. El servidor delega esta ráfaga a controladores especializados (como `mediaController.js`), los cuales actúan como nodos de conmutación puros: reciben la trama, consultan la topología de la sala en memoria y retransmiten de forma inmediata los *payloads* al conjunto de sockets válidos, garantizando fluidez sin esperar confirmaciones de la capa de aplicación.

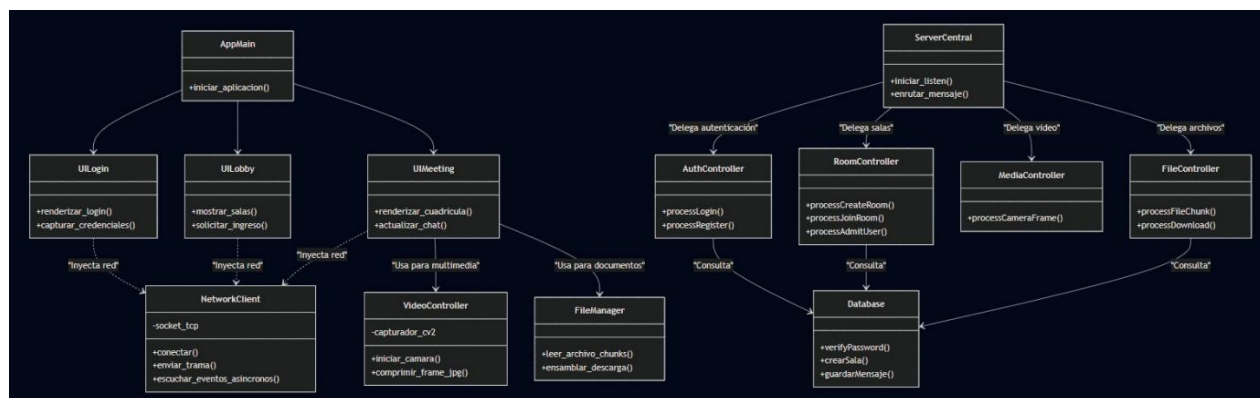


Figura 3: Diagrama de Clases

DISEÑO DE BASE DE DATOS Y PERSISTENCIA

Motor Relacional y Enfoque Físico

Para la capa de persistencia se seleccionó **SQLite** (a través del módulo `node:sqlite` en el servidor). La elección de un motor embebido responde a los requerimientos del prototipo, permitiendo una ejecución sincrónica en memoria y almacenamiento directo en un archivo local (`videoconferencia.db`). Esto reduce la sobrecarga de red interna (latencia `loopback`) y simplifica el despliegue del sistema sin requerir un servidor de base de datos independiente (como SQL Server o PostgreSQL), manteniendo el rigor transaccional y el cumplimiento de las propiedades ACID.

Modelo Entidad-Relación y Diccionario de Datos

El esquema relacional ha sido normalizado para evitar la redundancia de datos y garantizar la coherencia del estado de las reuniones. Está compuesto por las siguientes 6 entidades principales:

1. **Usuarios:** Entidad maestra de identidad. Almacena nombres, correo electrónico (definido como `UNIQUE` para evitar registros duplicados) y el `PasswordHash` cifrado mediante `PBKDF2`. Incluye una bandera booleana lógica (Activo).
2. **Salas:** Representa una instancia de reunión. Contiene el `CodigoSala` (identificador alfanumérico único para unirse) y establece una relación directa con el usuario creador a través de la llave foránea `IdHost`.
3. **ParticipantesSala:** Entidad pivote (tabla de rompimiento) que resuelve la relación de muchos a muchos entre `Usuarios` y `Salas`. Implementa una restricción de unicidad

compuesta `UNIQUE(IdSala, IdUsuario)` para impedir que un mismo usuario ingrese dos veces a la misma sesión.

4. **SolicitudesSala:** Estructura transitoria que registra el estado (Pendiente) de los usuarios invitados que esperan en el "Lobby" o sala de espera hasta que el host resuelva su admisión.
5. **Mensajes:** Repositorio cronológico del chat de texto. Cada registro está vinculado de forma inmutable a la sala donde se emitió y al usuario remitente, estampando la `FechaEnvio` mediante la función nativa `CURRENT_TIMESTAMP`.
6. **ArchivosCompartidos:** Tabla de metadatos de almacenamiento. No guarda el binario (BLOB), sino el nombre lógico (`NombreArchivo`) y el puntero de acceso en disco (`RutaArchivo`), optimizando el peso de la base de datos.

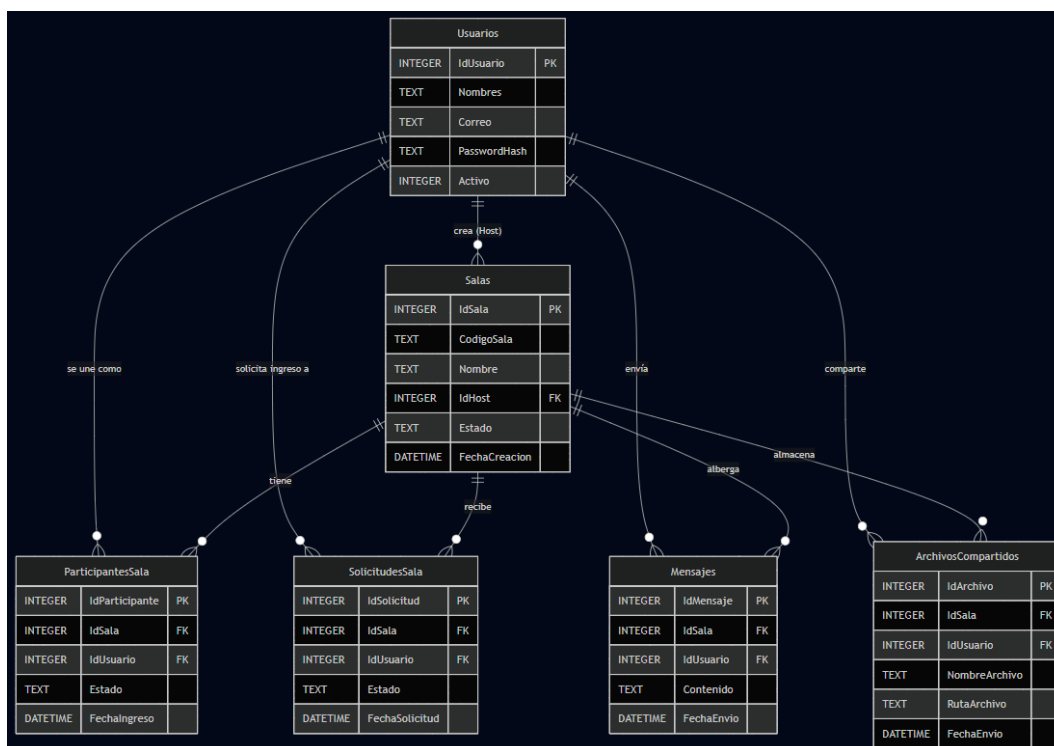


Figura 4: Modelo Relacional de la Base de Datos

Restricciones e Integridad Referencial

Para mantener la salud de los datos, el esquema implementa de manera estricta restricciones lógicas (Constraints):

- **Llaves Primarias (PK):** Autoincrementales (INTEGER PRIMARY KEY AUTOINCREMENT) para aislar la lógica de negocio de la indexación física.
- **Borrado en Cascada (ON DELETE CASCADE):** Todas las llaves foráneas (FK) han sido configuradas con propagación de eliminación. Si el administrador del sistema elimina un registro de la tabla Salas, automáticamente el motor SQLite destruirá todos los registros asociados en Mensajes, ParticipantesSala, SolicitudesSala y ArchivosCompartidos, previniendo datos huérfanos y violaciones de integridad.

Script de Definición de Datos (DDL)

A continuación, se adjunta el script SQL central (schema.sql) desarrollado para la instanciación automática del modelo físico:


```

1  -- Tabla de Usuarios --
2  ✓ CREATE TABLE IF NOT EXISTS Usuarios (
3      IdUsuario INTEGER PRIMARY KEY AUTOINCREMENT,
4      Nombres TEXT NOT NULL,
5      Correo TEXT NOT NULL UNIQUE,
6      PasswordHash TEXT NOT NULL,
7      Activo INTEGER NOT NULL DEFAULT 1
8  );
9
10 -- Tabla de Salas --
11 ✓ CREATE TABLE IF NOT EXISTS Salas (
12     IdSala INTEGER PRIMARY KEY AUTOINCREMENT,
13    CodigoSala TEXT NOT NULL UNIQUE,
14     Nombre TEXT NOT NULL,
15     IdHost INTEGER NOT NULL,
16     Estado TEXT NOT NULL DEFAULT 'Activa',
17     FechaCreacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
18     FOREIGN KEY (IdHost) REFERENCES Usuarios(IdUsuario) ON DELETE CASCADE
19 );
20
21 -- Tabla de Participantes en Sala --
22 ✓ CREATE TABLE IF NOT EXISTS ParticipantesSala (
23     IdParticipante INTEGER PRIMARY KEY AUTOINCREMENT,
24     IdSala INTEGER NOT NULL,
25     IdUsuario INTEGER NOT NULL,
26     Estado TEXT NOT NULL, -- 'Host' o 'Aceptado'
27     FechaIngreso DATETIME DEFAULT CURRENT_TIMESTAMP,
28     FOREIGN KEY (IdSala) REFERENCES Salas(IdSala) ON DELETE CASCADE,
29     FOREIGN KEY (IdUsuario) REFERENCES Usuarios(IdUsuario) ON DELETE CASCADE,
30     UNIQUE(IdSala, IdUsuario)
31 );

```

```

32
33 -- Tabla de Solicitudes Ingreso --
34 ∨ CREATE TABLE IF NOT EXISTS SolicitudesSala (
35     IdSolicitud INTEGER PRIMARY KEY AUTOINCREMENT,
36     IdSala INTEGER NOT NULL,
37     IdUsuario INTEGER NOT NULL,
38     Estado TEXT NOT NULL DEFAULT 'Pendiente',
39     FechaSolicitud DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
40     FOREIGN KEY (IdSala) REFERENCES Salas(IdSala) ON DELETE CASCADE,
41     FOREIGN KEY (IdUsuario) REFERENCES Usuarios(IdUsuario) ON DELETE CASCADE
42 );
43
44 -- Tabla de Mensajes --
45 ∨ CREATE TABLE IF NOT EXISTS Mensajes (
46     IdMensaje INTEGER PRIMARY KEY AUTOINCREMENT,
47     IdSala INTEGER NOT NULL,
48     IdUsuario INTEGER NOT NULL,
49     Contenido TEXT NOT NULL,
50     FechaEnvio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
51     FOREIGN KEY (IdSala) REFERENCES Salas(IdSala) ON DELETE CASCADE,
52     FOREIGN KEY (IdUsuario) REFERENCES Usuarios(IdUsuario) ON DELETE CASCADE
53 );
54
55 -- Tabla de Archivos Compartidos --
56 ∨ CREATE TABLE IF NOT EXISTS ArchivosCompartidos (
57     IdArchivo INTEGER PRIMARY KEY AUTOINCREMENT,
58     IdSala INTEGER NOT NULL,
59     IdUsuario INTEGER NOT NULL,
60     NombreArchivo TEXT NOT NULL,
61     RutaArchivo TEXT NOT NULL,
62     FechaEnvio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
63     FOREIGN KEY (IdSala) REFERENCES Salas(IdSala) ON DELETE CASCADE,
64     FOREIGN KEY (IdUsuario) REFERENCES Usuarios(IdUsuario) ON DELETE CASCADE
65 );
66

```

Inserción de Datos Base (Seed Data)

Para asegurar que el prototipo inicie con un entorno funcional demostrable, se configuró un proceso de inyección de datos de prueba en tiempo de arranque (seed.sql). Este proceso inserta cuentas con privilegios de anfitrión (Host), usuarios regulares e instancia una sala predeterminada (AULA123), de manera que el tribunal evaluador pueda realizar pruebas de validación cruzada sin necesidad de registros manuales previos.

PROTOCOLO DE COMUNICACIÓN POR SOCKETS (FRAMING)

Justificación del Modelo de Enmarcado (Framing)

El protocolo TCP es un flujo continuo de bytes (*byte-stream*), lo que significa que no preserva los límites de los mensajes originales. Si un cliente envía dos mensajes JSON rápidamente, el servidor podría recibirlos concatenados (`{msg1}{msg2}`), provocando un error fatal en el *parser*. Para solucionar este problema de frontera sin recurrir a protocolos de capa 7 como HTTP, se diseñó un **Protocolo de Enmarcado (Framing) Binario** personalizado.

Estructura de la Trama de Red

Cada transmisión entre el cliente y el servidor se empaqueta obligatoriamente en una trama estructurada que permite el envío simultáneo de metadatos de control (texto) y cargas pesadas (multimedia). La trama consiste en una **Cabecera Fija de 8 bytes** seguida de un cuerpo de longitud variable:

HEADER (8 bytes)		PAYLOAD VARIABLE	
JSON length	Binary length	JSON String (UTF- 8)	Binary Data Raw
4 bytes	4 bytes	Tamaño dinámico	Tamaño dinámico

- **JSON Length (UInt32 Big Endian):** Los primeros 4 bytes indican el tamaño exacto en bytes de la cadena JSON.
- **Binary Length (UInt32 Big Endian):** Los siguientes 4 bytes indican el tamaño exacto de la carga útil binaria (si no hay archivo o cámara, este valor es 0).
- **Decodificación:** El hilo receptor asíncrono (ahora encapsulado estrictamente dentro de `network_client.py` en el cliente Python) y el evento `data` del núcleo TCP (`server.js` en Node.js) leen primero la cabecera de 8 bytes. Se decodifican los tamaños y se espera en el

buffer exactamente esa cantidad de bytes antes de reconstruir el paquete. Una vez validada la trama, el cliente la envía a la interfaz gráfica y el servidor la delega a la capa de Controladores. El tamaño máximo de trama está limitado a 10 MB (MAX_FRAME_SIZE).

Tipos de Mensajes y Serialización JSON

La capa de aplicación utiliza JSON para enrutar las acciones. A continuación, se detallan los principales comandos de comunicación bidireccional implementados:

Mensajes de Control de Sesión y Salas:

- LOGIN_REQUEST / LOGIN_RESPONSE: Autenticación de credenciales.
- CREATE_ROOM / CREATE_ROOM_RESPONSE: Inicialización de una instancia de videollamada.
- JOIN_ROOM_REQUEST / JOIN_ROOM_RESPONSE: Petición de un invitado para unirse a una sesión activa.
- WAITING_ROOM_UPDATE: Notificación asíncrona enviada al *Host* con la lista de usuarios pendientes.
- ADMIT_USER / ADMIT_RESULT: Decisión de control de acceso emitida por el *Host*.

Mensajes de Interacción de la Reunión:

- CHAT_MESSAGE: Transporte de texto plano. Incluye userId, userName, message y sentAt.

- **FILE_CHUNK**: Transmisión fraccionada de documentos. Contiene índice del bloque y un flag isLast. Se acompaña obligatoriamente del *Payload Binario*.
- **CAMERA_FRAME**: Notificación de un nuevo fotograma disponible. La imagen JPG comprimida viaja adjunta en el *Payload Binario*.
- **PARTICIPANTS_UPDATE**: Sincronización del estado de la sala para re-renderizar la cuadrícula de videos en los clientes.

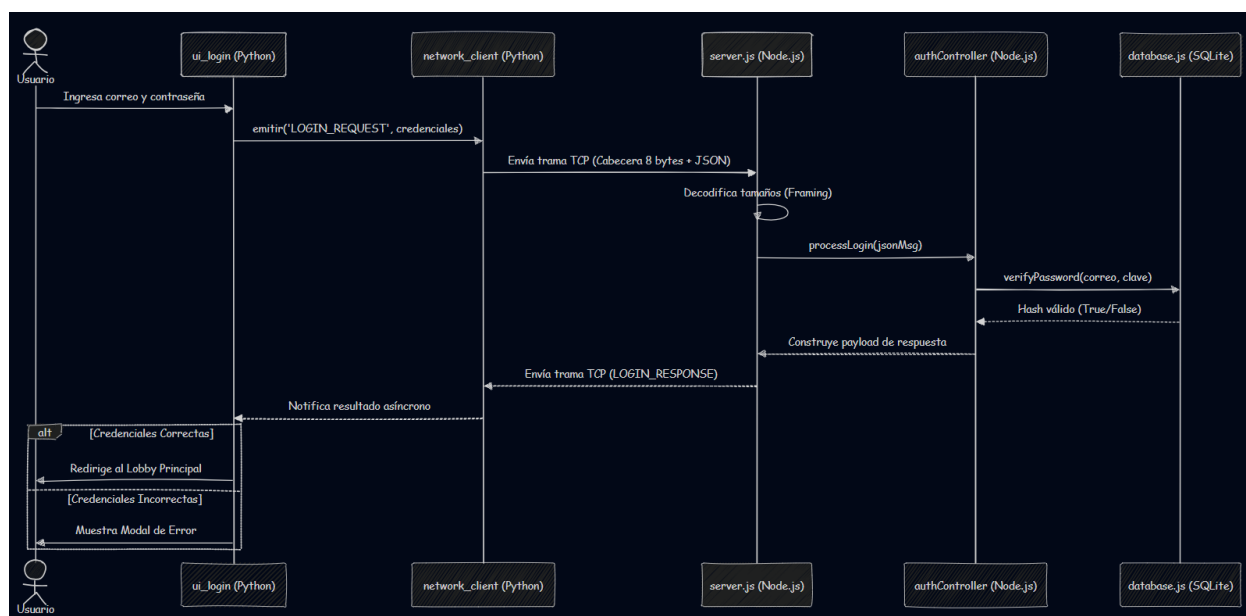


Figura 5: Diagrama de Secuencia

IMPLEMENTACIÓN DE MÓDULOS

La construcción del sistema se ha estructurado en torno a cinco módulos funcionales críticos. Cada uno de estos módulos implementa de manera estricta el ciclo de petición-respuesta y la retransmisión asíncrona de flujos de datos sobre el protocolo TCP, respetando el enmarcado de capa de aplicación (Framing) previamente definido.

Consideración Arquitectónica Transversal

Bajo el paradigma de diseño actual, la ejecución de todos los módulos obedece a un flujo de trabajo altamente desacoplado. En la capa de presentación del cliente (directorio /UI), se instancian los eventos de usuario, los cuales son delegados a la capa lógica subyacente (a través de `network_client.py` y controladores multimedia) para su serialización. Al alcanzar el backend, el núcleo de red del servidor actúa como un *Dispatcher*: intercepta, decodifica y enruta la petición exclusivamente hacia su controlador de dominio correspondiente (ubicado en el directorio /Controllers). Este modelo garantiza el aislamiento de la lógica de negocio respecto a la capa de transporte, asegurando el cumplimiento del Principio de Responsabilidad Única (SRP) en cada transacción.

Módulo de Login e Identidad

La autenticación protege el acceso al sistema validando credenciales contra la base de datos antes de conceder una sesión de red.

- **Cliente:** La interfaz gráfica (GUI) captura el correo y la contraseña mediante componentes `CTkEntry`. Al solicitar el ingreso, se bloquean los controles visuales y se

instancia un hilo secundario que intenta la conexión de red. Si el socket se establece, emite la trama LOGIN_REQUEST.

- **Servidor:** El módulo authController.js intercepta la petición y delega la ejecución de la función criptográfica nativa de Node.js (crypto.pbkdf2Sync) para generar el hash de la contraseña entrante y compararlo en tiempo constante con el hash almacenado en SQLite. Si es válido, se mapea el IdUsuario al socket dentro de la estructura en memoria socketSessions, otorgándole estado de "autenticado".

Módulo de Salas y Sala de Espera (Control de Acceso)

La gestión de concurrencia agrupa a los usuarios en sesiones aisladas (Salas) y otorga privilegios de administración al creador (Anfitrión / Host).

- **Flujo de Ingreso:** Cuando un invitado envía un JOIN_ROOM_REQUEST, el servidor no lo une directamente a la videollamada. En su lugar, lo inserta en un arreglo temporal asociado al código de la sala (waitingRooms) y emite una trama WAITING_ROOM_UPDATE al socket del anfitrión.
- **Aprobación:** El cliente del anfitrión recibe la actualización y renderiza dinámicamente una lista de solicitudes pendientes en su panel lateral. Al hacer clic en "Admitir", se emite un comando ADMIT_USER. El módulo roomController.js procesa esta orden transaccional, traslada el socket del invitado al conjunto principal de la sala (activeRooms) y le envía el historial de chat y archivos previos, concretando su ingreso a la reunión.

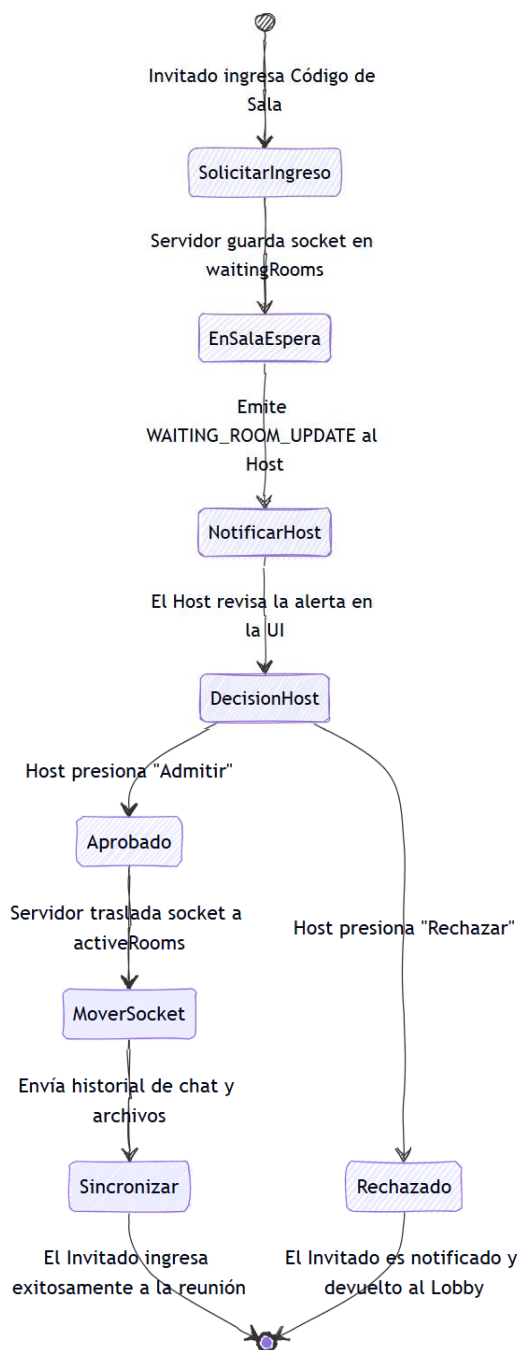


Figura 6: Diagrama de Actividades

Módulo de Chat de Texto

Implementa mensajería persistente en tiempo real mediante un esquema de difusión (Broadcast).

- **Cliente:** El usuario escribe en la entrada inferior y al presionar "Enviar", se despacha una trama `CHAT_MESSAGE` que incluye el contenido y la marca de tiempo local.
- **Servidor:** Al recibir el mensaje (`processChatMessage`), el sistema primero realiza una inserción sincrónica en la tabla Mensajes de la base de datos (`guardarMensaje`) para asegurar la persistencia. Inmediatamente después, itera sobre el conjunto de sockets de la sala emisora y retransmite el paquete íntegro. Los clientes receptores actualizan su `CTkTextbox` e invocan una alerta sonora discreta usando la librería `winsound`.

Módulo de Transferencia de Documentos (I/O en Bloques)

Para enviar archivos sin bloquear el único hilo del socket y sin desbordar la memoria RAM (tanto en Python como en Node.js), se programó un mecanismo de subida y descarga incremental.

- **Subida (Upload):** El cliente selecciona un archivo local. El módulo lógico `file_manager.py` administra un hilo en segundo plano que fracciona el archivo matemáticamente en fragmentos (chunks) de 4 Kilobytes. Cada fragmento se envía recursivamente como un `FILE_CHUNK` binario, permitiendo a la GUI actualizar una barra de progreso. El controlador `fileController.js` recibe estos fragmentos en orden y los concatena físicamente en el disco duro usando `fs.createWriteStream()`. Al llegar el último chunk, se guarda el metadato en la base de datos y se notifica a la sala.
- **Descarga (Download):** Cuando un usuario solicita descargar, envía un `FILE_DOWNLOAD_REQUEST`. El servidor abre el archivo desde el directorio estático usando `fs.createReadStream()` con un *highWaterMark* de 16KB, enviando los bytes fraccionados de vuelta al cliente, quien los reconstruye en su directorio de destino local.

Módulo de Captura y Transmisión de Cámara (Procesamiento de Imágenes)

Para cumplir con la restricción académica de no saturar la red local con protocolos inadecuados para multimedia sobre TCP, la transmisión de video se optimizó severamente mediante ráfagas de imágenes.

- **Captura y Compresión:** El controlador `video_controller.py` aísla la ejecución de la librería OpenCV en un bucle continuo sobre un hilo secundario (`_camera_capture_loop`). Se captura un fotograma, se escala a una resolución mínima de 320x240 píxeles, se convierte su espacio de color (BGR a RGB) y se codifica a formato JPEG forzando una degradación de calidad al 50%.
- **Distribución:** La ráfaga binaria resultante es delegada al servidor, donde el `mediaController.js` actúa como espejo reflectante, desviando los bytes crudos a todos los sockets de la sala, excepto al emisor.
- **Renderizado:** Al recibir la imagen, el cliente receptor utiliza Pillow (PIL) e `io.BytesIO` para decodificar los bytes en memoria y actualizar dinámicamente la propiedad de imagen del componente visual `CTkImage` asignado a dicho usuario en la cuadrícula principal.

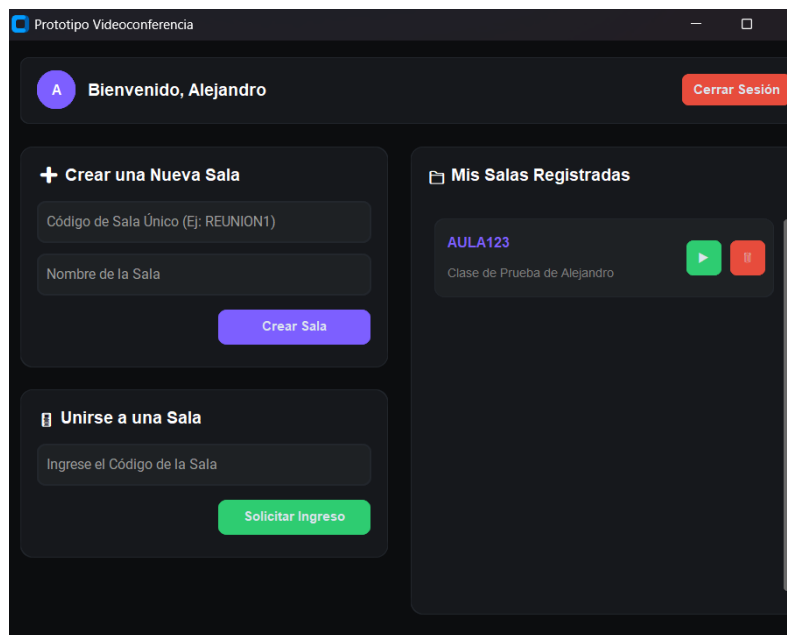
PRUEBAS Y CASOS DE USO

Para validar la robustez del prototipo y garantizar el cumplimiento de los requerimientos mínimos establecidos por la cátedra, se ejecutó un plan de pruebas en un entorno de red local. A continuación, se detallan los casos de prueba (CP) obligatorios y sus respectivas evidencias.

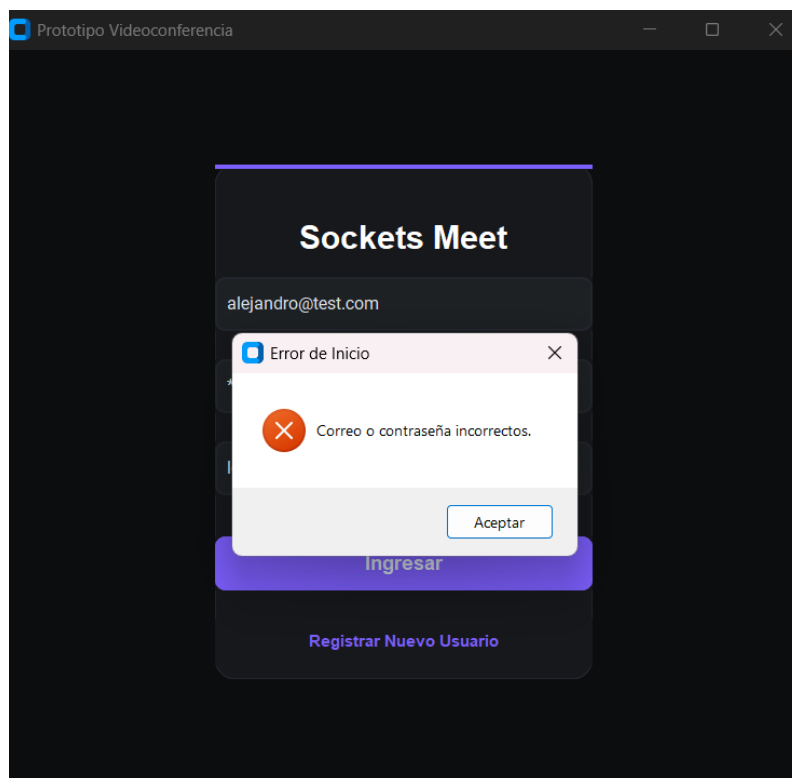
Pruebas de Autenticación y Control de Sesión

ID	Caso de Prueba	Precondiciones	Resultado Esperado	Estado
CP-01	Ingreso con usuario válido	El usuario "Alejandro" existe en SQLite con contraseña correcta.	El sistema genera el token de sesión y redirige al panel de control (Lobby), saludando al usuario.	ÉXITO
CP-02	Ingreso con credenciales erróneas	Se ingresa una contraseña que no coincide con el hash almacenado.	El servidor rechaza la conexión, el GUI muestra un modal de error sin colapsar la aplicación.	ÉXITO

- CP-01:



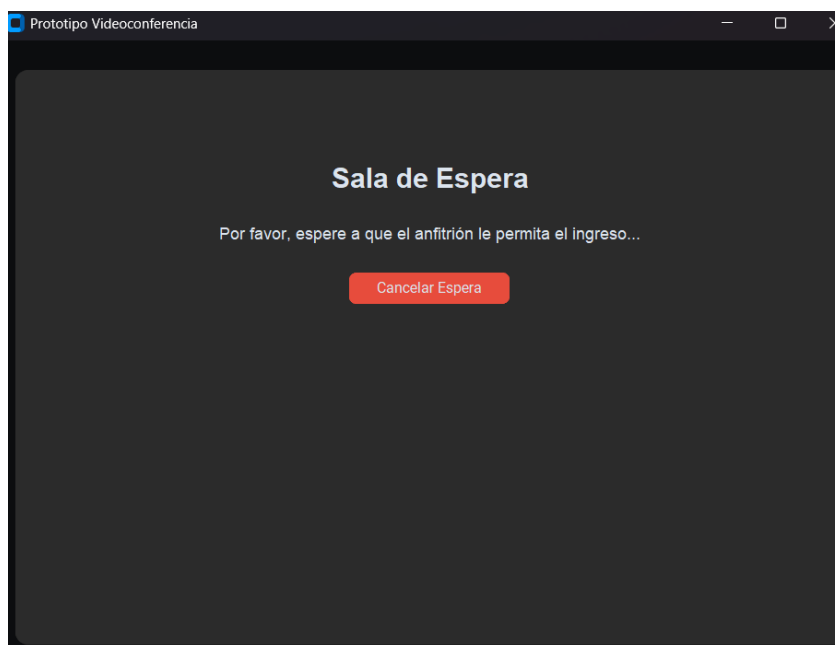
- CP-02:



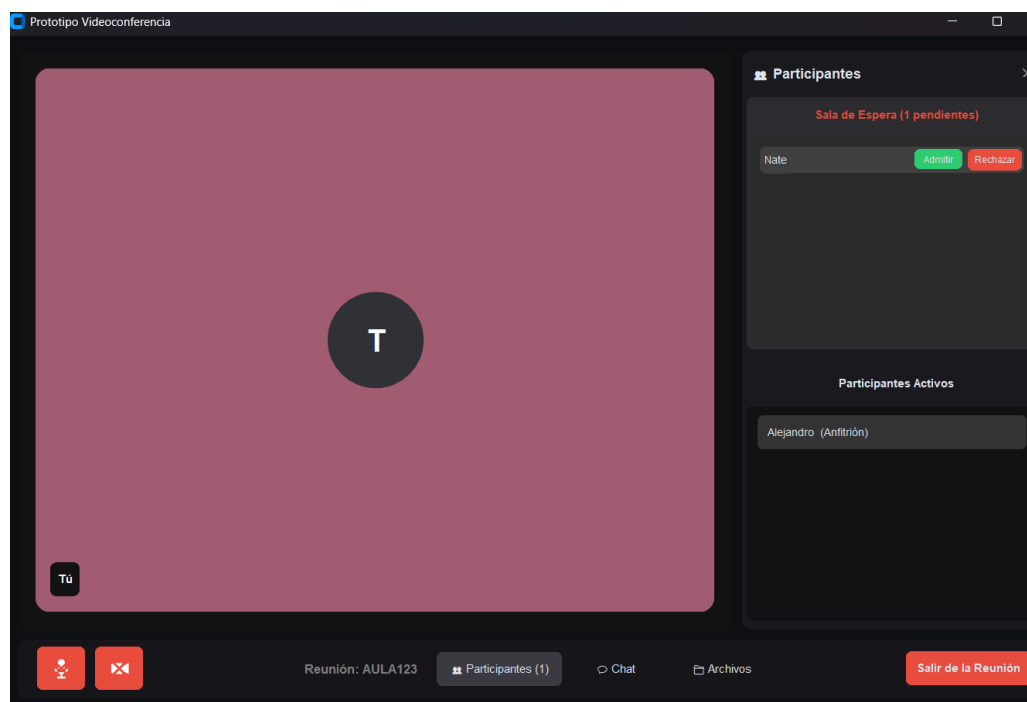
Pruebas de Salas y Gestión de Concurrencia

ID	Caso de Prueba	Precondicione s	Resultado Esperado	Estado
CP-03	Host crea una sala	Usuario autenticado genera un código y nombre para una sala.	El código "AULA123" se registra en la BD y aparece en la lista de salas activas del usuario.	ÉXITO
CP-04	Invitado solicita entrar	El invitado "Nate" usa el código "AULA123".	El invitado queda retenido en una pantalla oscura de "Sala de Espera".	ÉXITO
CP-05	Host acepta al invitado	El anfitrión "Alejandro" recibe notificación de "1 pendiente" en su panel lateral.	Al presionar "Admitir", el invitado ingresa a la reunión y se renderizan ambas cuadrículas de video.	ÉXITO

- CP-04:



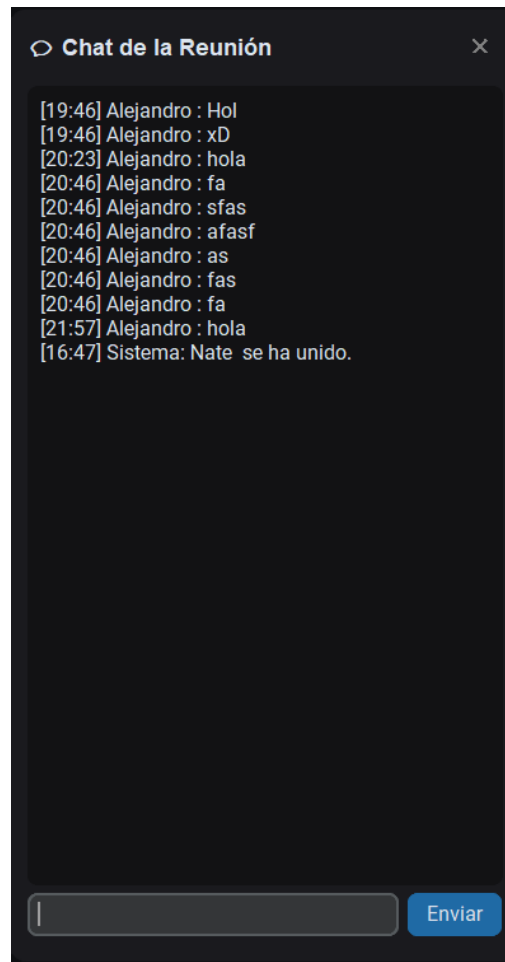
- CP-03 y CP-05:



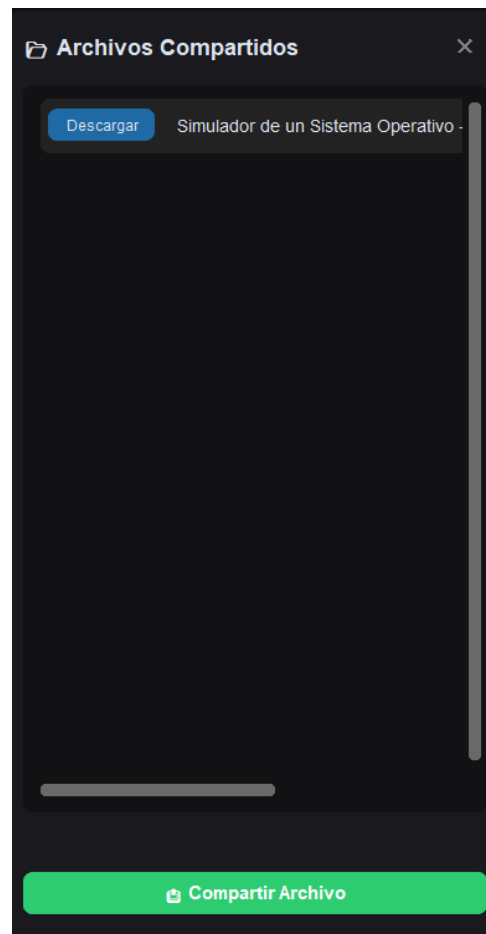
Pruebas de Interacción Asíncrona (Chat y Multimedia)

ID	Caso de Prueba	Precondiciones	Resultado Esperado	Estado
CP-06	Enviar mensaje de chat	Dos usuarios activos en la misma sala ("AULA123").	El chat se sincroniza en ambas pantallas, mostrando las marcas de tiempo y el remitente.	ÉXITO
CP-07	Compartición de documentos	El host selecciona un documento en disco.	El documento se fracciona, viaja por red y aparece en la pestaña "Archivos Compartidos" listo para descarga.	ÉXITO
CP-08	Transmisión de Cámara Local	El usuario activa el botón de cámara en la barra de controles.	La cámara se enciende mediante OpenCV, se comprime en JPG y se visualiza en los lienzos de los demás participantes.	ÉXITO

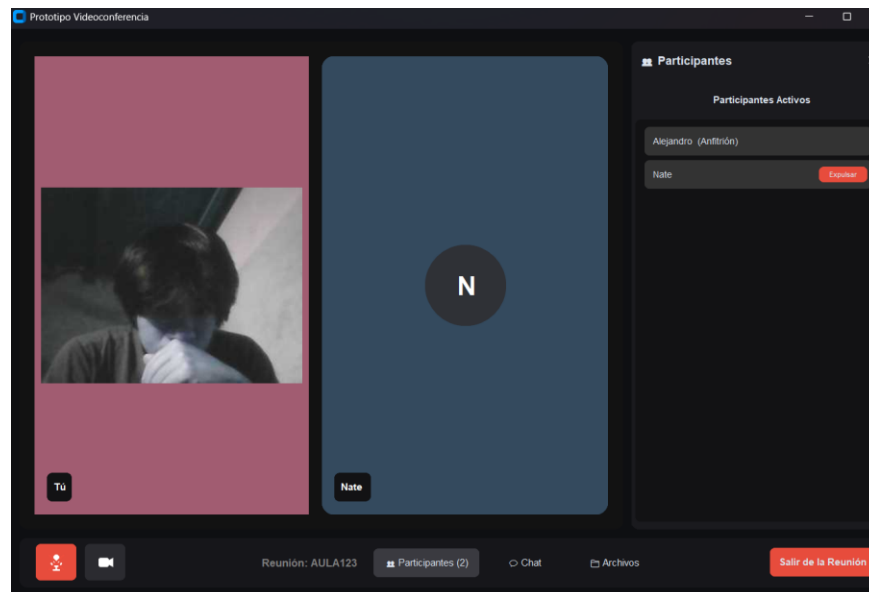
- CP-06



- CP-07



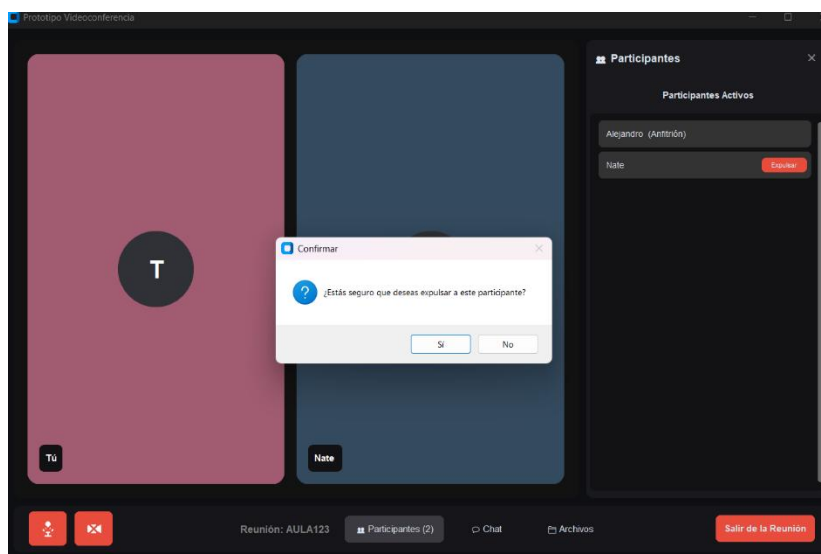
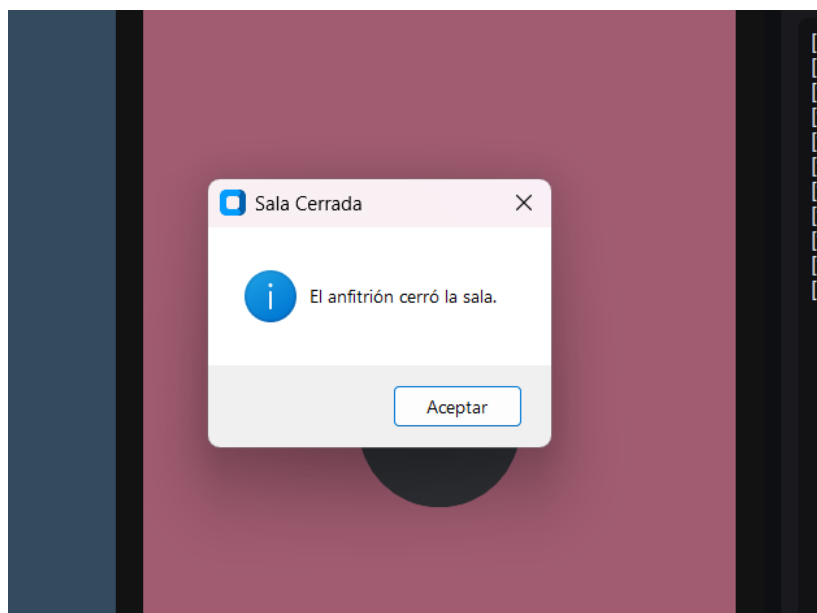
- CP-08

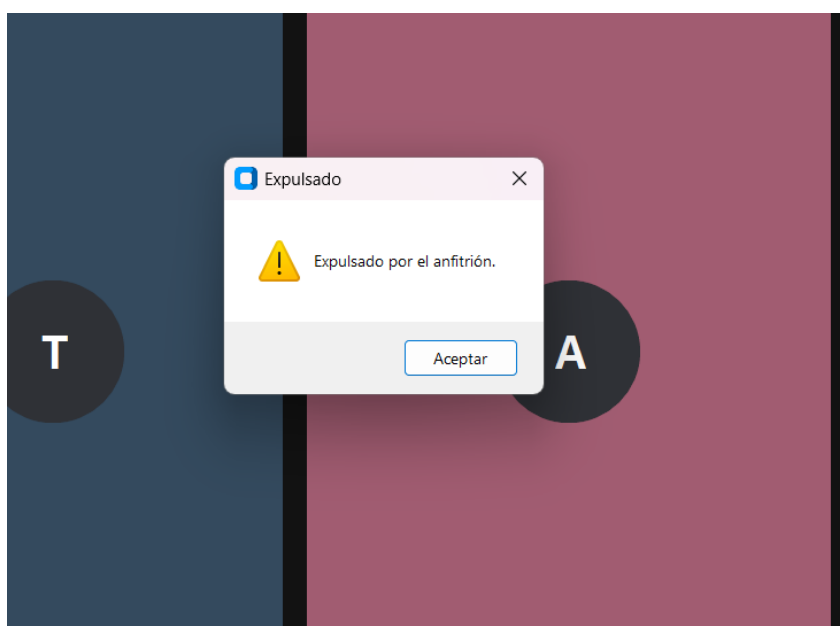
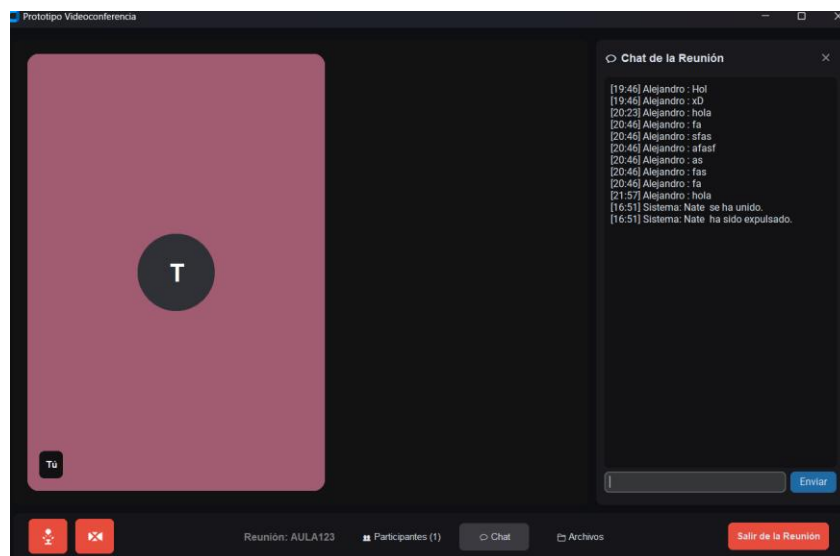


Pruebas de Control Administrativo (Opcionales / Mejoras)

Como mejora sustancial a los requisitos mínimos, se validaron las directivas de cierre de sesión forzado por parte del administrador.

ID	Caso de Prueba	Precondiciones	Resultado Esperado	Estado
CP-10	Cierre global de la sala	El anfitrión presiona "Salir de la Reunión".	Se desconecta el socket central, todos los invitados reciben un modal informando el cierre y son devueltos al Lobby.	ÉXITO
CP-11	Expulsión de un participante	El anfitrión usa la pestaña "Participantes" para expulsar a un usuario específico.	El usuario es desconectado con una alerta de expulsión, y el anfitrión recibe notificación en el chat de que fue retirado.	ÉXITO





CONCLUSIONES Y MEJORAS FUTURAS

Conclusiones

- 1. Viabilidad de Arquitecturas Asíncronas Básicas:** Se demostró la viabilidad técnica de construir un sistema de comunicación en tiempo real utilizando Sockets TCP puros en combinación con hilos (threading) orientados a eventos, logrando una sincronización estable entre múltiples clientes sin colapsar la interfaz gráfica de usuario.
- 2. Resolución de Fronteras TCP (Framing):** La implementación de una cabecera de 8 bytes demostró ser una solución arquitectónica sólida para el problema intrínseco de TCP (*byte-stream*), permitiendo la lectura determinista y la correcta separación entre metadatos (JSON) y cargas binarias pesadas.
- 3. Gestión de Memoria en Transferencias:** El fraccionamiento de archivos en bloques de 4KB (Chunks) garantizó un uso de memoria (RAM) constante y seguro en el servidor Node.js, validando que el procesamiento de streams es fundamental para evitar cuellos de botella en operaciones de entrada/salida (I/O).
- 4. Aislamiento de la Persistencia:** La arquitectura de tres capas cumplió su objetivo de seguridad, aislando totalmente a los clientes del motor SQLite, lo que evita inyecciones directas de SQL y centraliza la lógica de validación (como el cifrado PBKDF2) en el servidor.
- 5. Modularidad y Separación de Responsabilidades (SRP):** La refactorización del código base hacia un patrón Modelo-Vista-Controlador (MVC) en el cliente y una arquitectura basada en Controladores y DAO en el servidor demostró ser fundamental para erradicar el acoplamiento excesivo (antipatrón de código espagueti). Esta segmentación lógica probó que aislar la interfaz gráfica, el transporte de red y el acceso a datos no solo facilita

la mantenibilidad del prototipo, sino que permite que múltiples desarrolladores escalen el sistema de manera independiente sin riesgo de colapsar la arquitectura central.

Dificultades Encontradas y Limitaciones del Prototipo

- 1. Latencia Multimedia sobre TCP:** TCP garantiza la entrega en orden mediante confirmaciones (ACKs) y retransmisiones. Sin embargo, para flujos multimedia en tiempo real (cámara web), esta característica introduce latencia y congelamientos temporales (*Head-of-Line Blocking*) cuando hay pérdida de paquetes en la red.
- 2. Seguridad en el Transporte:** Actualmente, las tramas TCP (JSON y binarios) viajan en texto plano a través de la red local, lo cual es vulnerable a interceptación mediante analizadores de paquetes (Sniffers).

Mejoras Futuras Recomendadas

Para escalar este prototipo académico hacia un estándar comercial o de producción, se plantean las siguientes evoluciones arquitectónicas:

- 1. Migración de Transporte Multimedia:** Sustituir la ráfaga de imágenes TCP por el protocolo UDP combinado con el estándar **WebRTC** para habilitar una transmisión de video y audio bidireccional de baja latencia y alta definición.
- 2. Seguridad y Cifrado (TLS / E2EE):** Envolver el socket TCP puro en una capa de Seguridad de la Capa de Transporte (TLS) para cifrar los metadatos y el chat, evaluando la implementación de cifrado de extremo a extremo (End-to-End Encryption) para los mensajes privados.

3. **Motor de Base de Datos Concurrente:** Migrar de SQLite (orientado a un solo archivo con bloqueo de escritura global) a un motor relacional escalable como PostgreSQL o SQL Server, permitiendo una mayor concurrencia de inserciones en salas masivas.

ANEXOS

El código fuente íntegro de la solución, incluyendo el aplicativo cliente, el servidor de sockets y los scripts de inicialización de la base de datos, se encuentra versionado y alojado en el siguiente repositorio de GitHub:

- **Enlace al Repositorio Oficial:** <https://github.com/AlejandroAVT/Socketts>